

MÉMO DE RÉVISION

Harbor + Terraform + Pipeline CI sécurisée

TP DevSecOps — préparation à l'oral

IaC (Terraform/Helm) · CI sécurisée (lint·SAST·CVE·DAST·signature) · Observabilité

Dylan TREPON

Dépôt : github.com/KaTaKuRi-31/harbor-devsecops

Sommaire

Sommaire.....	2
1. Vue d'ensemble : de quoi parle ce projet ?.....	4
1.1. Schéma d'architecture.....	4
1.2. Le vocabulaire de base (à maîtriser).....	6
2. Pilier 1 — L'Infrastructure as Code (Terraform)	7
2.1. Le problème résolu.....	7
2.2. Les trois outils et leur rôle	7
2.3. Les commandes essentielles	7
2.4. Le fichier central : terraform/main.tf, expliqué	7
2.5. Les autres fichiers du dossier terraform/	8
3. Pilier 2 — La pipeline CI sécurisée (DevSecOps).....	9
3.1. L'idée : « shift-left security »	9
3.2. Les cinq portes de sécurité.....	9
3.3. Le mot-clé « bloquant » : exit-code 1.....	9
3.4. La signature Cosign « keyless »	9
3.5. Runners hébergés vs self-hosted (LA question piège)	10
3.6. Le fichier .github/workflows/ci.yml, job par job	10
3.7. Le parcours complet d'une image	10
4. Pilier 3 — L'observabilité (Prometheus + Grafana)	12
4.1. Le principe	12
4.2. La chaîne de collecte	12
4.3. Pourquoi un exporter CVE « maison » ?.....	12
4.4. Le dashboard Grafana « Harbor — Registre & CVE ».....	12
4.5. Requêtes PromQL à connaître.....	13
5. Les autres fichiers clés à savoir expliquer	14
5.1. app/Dockerfile (l'image de démonstration).....	14
5.2. app/app.py	14
5.3. terraform/cve-exporter.tf	14
5.4. scripts/start-demo.ps1 et stop-demo.ps1.....	14
6. Manipulations : « le prof demande... »	15
6.1. Commandes de manipulation prêtes à l'emploi	15

7. Questions / réponses probables du jury	16
8. Glossaire express	17

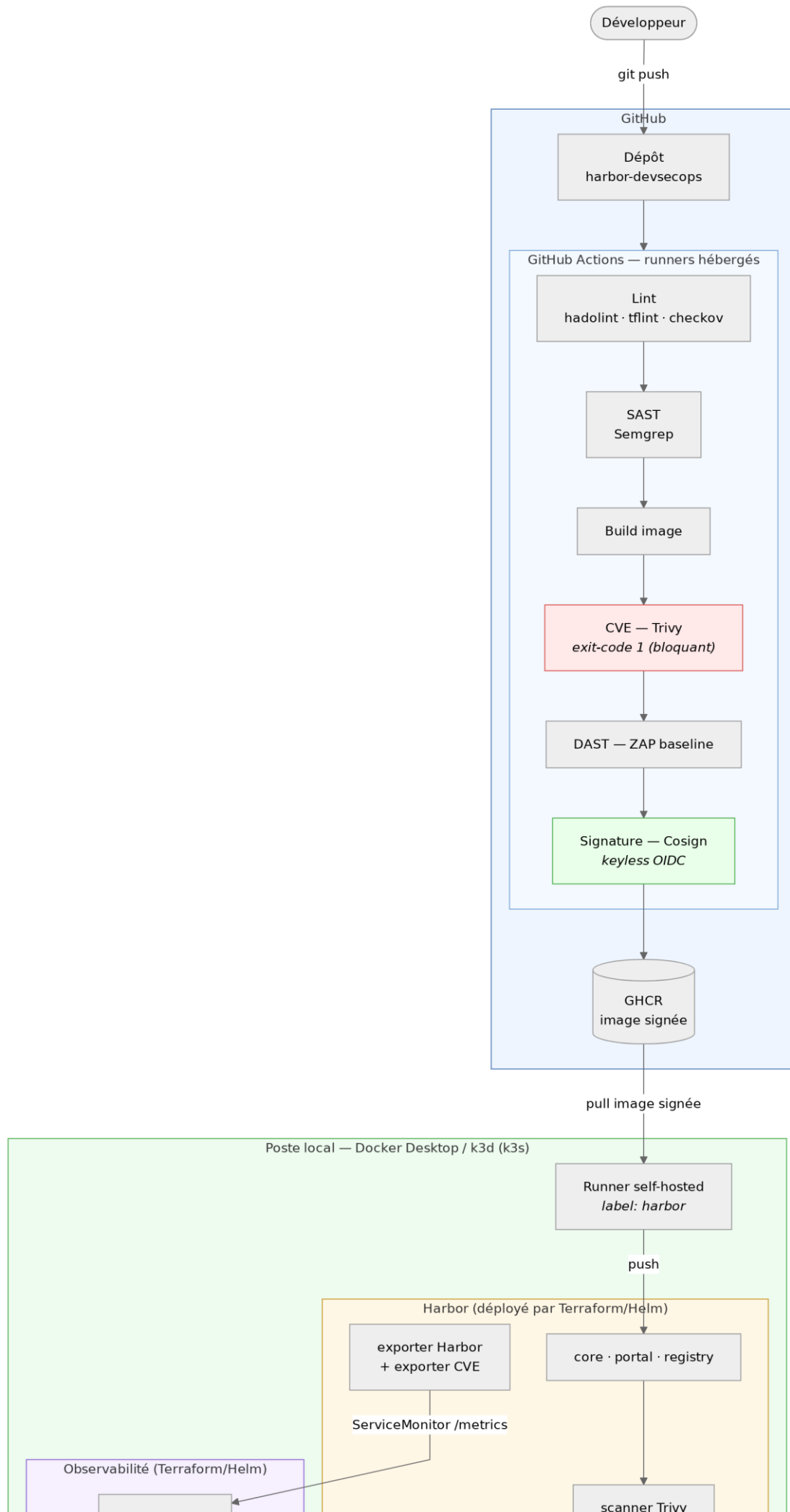
1. Vue d'ensemble : de quoi parle ce projet ?

Le projet met en place, de façon entièrement automatisée, un **registre d'images Docker d'entreprise (Harbor)**, et tout l'écosystème de sécurité et de surveillance autour. Quatre briques s'enchaînent :

- **Un entrepôt d'images** : Harbor stocke les images Docker (les applications empaquetées) et les scanne à la recherche de vulnérabilités.
- **Une installation automatique (IaC)** : Terraform + Helm déploient Harbor et la surveillance sur un mini-Kubernetes (k3s/k3d), de façon reproductible.
- **Une chaîne de contrôle (CI sécurisée)** : à chaque modification du code, GitHub Actions passe une série de portes de sécurité (lint, SAST, CVE, DAST, signature) ; si une porte échoue, la livraison est bloquée.
- **La surveillance (observabilité)** : Prometheus collecte les métriques de Harbor (et les vulnérabilités), Grafana les affiche dans un tableau de bord.

Phrase d'ouverture pour l'oral : « J'ai déployé un registre Harbor par Infrastructure as Code, sécurisé sa chaîne de livraison avec une pipeline DevSecOps bloquante, et rendu le tout observable via Prometheus et Grafana. »

1.1. Schéma d'architecture



Architecture : du push développeur jusqu'à Harbor, en passant par les contrôles de sécurité et la publication.

1.2. Le vocabulaire de base (à maîtriser)

Terme	Définition simple
Conteneur / image Docker	Une application emballée avec tout ce qu'il lui faut pour tourner. L'« image » est le modèle, le « conteneur » est l'image en cours d'exécution.
Registre d'images	Un entrepôt où l'on stocke et distribue les images Docker (ex. Docker Hub, GHCR, et ici Harbor).
Kubernetes (k8s)	L'orchestrateur qui fait tourner et gère les conteneurs à grande échelle. k3s en est une version allégée.
k3d	Un outil qui lance un cluster k3s à l'intérieur de Docker (idéal sur un poste de travail).
Helm	Le « gestionnaire de paquets » de Kubernetes : installe des applications via des paquets paramétrables appelés charts.
Terraform	L'outil d'Infrastructure as Code : on décrit l'infra dans des fichiers, et une commande la crée.
CI/CD	Intégration / livraison continue : automatiser la construction, les tests et la livraison à chaque changement de code.

2. Pilier 1 — L'Infrastructure as Code (Terraform)

2.1. Le problème résolu

Installer Harbor « à la main » demanderait des dizaines de commandes, non reproductibles et sources d'erreurs. L'**IaC décrit l'infrastructure dans des fichiers texte**, versionnés dans Git, appliqués d'une seule commande. On obtient une infrastructure **reproductible, documentée et vérifiable**.

2.2. Les trois outils et leur rôle

Outil	Rôle (analogie)
k3s (via k3d)	Le cluster Kubernetes qui héberge tout. Analogie : le terrain.
Helm	Installe Harbor et la surveillance via des charts. Analogie : des meubles en kit.
Terraform	Orchestre le tout : pilote Helm pour installer les charts sur le cluster. Analogie : la notice de montage automatique.

À retenir : on ne « tape » pas l'installation, on la **déclare**. La commande `terraform apply` lit les fichiers et construit l'infrastructure.

2.3. Les commandes essentielles

```
cd terraform
terraform init      # télécharge les providers (helm, kubernetes)
terraform plan      # montre ce qui VA être créé/modifié (sans rien faire)
terraform apply     # applique réellement -> Harbor s'installe
terraform state list # liste tout ce qui est géré par l'IaC
```

2.4. Le fichier central : terraform/main.tf, expliqué

C'est LE fichier que l'on te demandera d'expliquer. Rappel de syntaxe : un bloc resource "TYPE" "NOM" { ... } déclare une ressource. Le préfixe du type (ex. helm_, kubernetes_) indique le **provider** utilisé. var.xxx renvoie à une variable de variables.tf.

a) Les namespaces

```
resource "kubernetes_namespace" "monitoring" { ... }
resource "kubernetes_namespace" "harbor"      { ... }
```

Un **namespace** est une cloison logique dans le cluster pour isoler des ressources. On en crée deux : monitoring (Prometheus/Grafana) et harbor (le registre). Les labels sont de simples étiquettes de documentation.

b) La release Helm de la surveillance

```
resource "helm_release" "kube_prometheus_stack" {
  name      = "kube-prometheus-stack"
  repository = "https://prometheus-community.github.io/helm-charts"
  chart     = "kube-prometheus-stack"
  version   = var.kps_chart_version
  namespace = kubernetes_namespace.monitoring.metadata[0].name
  values    = [ templatefile("../kube-prometheus-stack.yaml.tftpl", {...}) ]
}
```

```
set_sensitive { name = "grafana.adminPassword" value = var.grafana_admin_password }
timeout = 900 wait = true
}
```

- `helm_release` = « installe un chart Helm ».
- `repository / chart / version` = d'où vient le paquet, lequel, et quelle version (épinglée → reproductible).
- `namespace = kubernetes_namespace.monitoring...` : en référençant le namespace, Terraform comprend qu'il doit le créer AVANT (dépendance implicite). Le `[0]` vient du fait que `metadata` est une liste.
- `values = [ templatefile(...) ]` : les réglages Helm sont externalisés dans un fichier YAML, dans lequel `templatefile` injecte des variables (les ports). `.tftpl` = Terraform template.
- `set_sensitive` : injecte une valeur secrète (mot de passe) en la **masquant** dans les logs.
- `timeout = 900 / wait = true` : attendre jusqu'à 15 min que tous les pods soient prêts.

c) La release Helm de Harbor (+ le `depends_on` clé)

```
resource "helm_release" "harbor" {
  ... chart = "harbor", repository = "https://helm.goharbor.io" ...
  set_sensitive { name = "harborAdminPassword" value = var.harbor_admin_password }
  depends_on = [helm_release.kube_prometheus_stack] # <-- IMPORTANT
}
```

Pourquoi le `depends_on` ? Harbor crée un objet `ServiceMonitor` (l'objet qui dit à Prometheus de venir le surveiller). Or ce type d'objet est une **CRD** (Custom Resource Definition = un nouveau type d'objet ajouté à Kubernetes) **apportée par la stack Prometheus**. Il faut donc installer Prometheus en premier. Le `depends_on` force cet ordre.

2.5. Les autres fichiers du dossier `terraform/`

Fichier	Rôle
<code>versions.tf</code>	Versions de Terraform et des providers (helm, kubernetes).
<code>providers.tf</code>	Connexion au cluster k3d (via le kubeconfig, contexte k3d-harbor).
<code>variables.tf</code>	Tous les paramètres : mots de passe, ports, versions des charts.
<code>values/harbor.yaml.tftpl</code>	Réglages Harbor : exposition NodePort, scanner Trivy activé, métriques + ServiceMonitor activés.
<code>values/kube-prometheus-stack.yaml.tftpl</code>	Réglages Prometheus/Grafana : NodePort, sélection de tous les ServiceMonitor, rendre Grafana.
<code>cve-exporter.tf</code>	Déploiement de l'exporter CVE maison (voir Pilier 3).
<code>grafana-dashboard.tf</code>	Charge le dashboard Grafana via une ConfigMap.
<code>outputs.tf</code>	Affiche les URL utiles après le déploiement.

3. Pilier 2 — La pipeline CI sécurisée (DevSecOps)

3.1. L'idée : « shift-left security »

En DevSecOps, la sécurité n'est pas faite à la fin mais **intégrée dans la chaîne de livraison, au plus tôt** (« décaler à gauche »). À chaque push, une pipeline automatique exécute des contrôles de sécurité. Si l'un échoue, **la livraison est bloquée**. C'est la différence entre DevOps (livrer vite) et DevSecOps (livrer vite ET sûr). Le fichier `.github/workflows/ci.yml` décrit cette pipeline, exécutée par **GitHub Actions**.

3.2. Les cinq portes de sécurité

#	Porte	Outil	Ce qui est inspecté
1	Lint	hadolint · tflint · checkov	Qualité/sécurité du Dockerfile et du code d'infrastructure (IaC).
2	SAST	Semgrep	Le code source lui-même, analysé sans l'exécuter (failles de programmation).
3	CVE	Trivy	Les vulnérabilités connues des dépendances et de l'OS de l'image.
4	DAST	OWASP ZAP	L'application en cours d'exécution, attaquée de l'extérieur.
5	Signature	Cosign	Signe l'image pour prouver son origine (chaîne d'approvisionnement).

Les sigles : **SAST** = Static Application Security Testing (analyse statique, on lit le code). **DAST** = Dynamic (on lance l'app et on l'attaque). **CVE** = Common Vulnerabilities and Exposures (failles publiques répertoriées ; Trivy compare les paquets de l'image à cette base).

3.3. Le mot-clé « bloquant » : exit-code 1

Une étape « bloque » quand elle renvoie un code d'erreur qui stoppe la pipeline. Pour Trivy :

```
trivy image --severity HIGH,CRITICAL --ignore-unfixed --exit-code 1 <image>
```

- `--severity HIGH,CRITICAL` : on ne s'intéresse qu'aux failles graves.
- `--ignore-unfixed` : on ne bloque que sur les failles **corrigeables** (actionnables). Bloquer sur une faille sans correctif serait inutile.
- `--exit-code 1` : si une telle faille est trouvée → code 1 → pipeline rouge → image non livrée.

Démonstration imparable : Trivy sur l'app à jour renvoie **0** (vert) ; sur une image ancienne (python:3.9-slim-bullseye, 7 CVE Critical) renvoie **1** (rouge). C'est cela, une porte bloquante.

3.4. La signature Cosign « keyless »

Signer une image garantit qu'elle vient bien de toi et n'a pas été altérée (sécurité de la **supply chain**). **Keyless** = **sans clé privée** à gérer (donc rien à voler) : l'identité du signataire est le **workflow GitHub** lui-même, prouvée par **OIDC**. Le certificat est émis par **Fulcio** et la preuve inscrite dans le journal public **Rekor**. La commande `cosign verify` permet à quiconque de vérifier que l'image vient bien de ce dépôt et de ce workflow.

3.5. Runners hébergés vs self-hosted (LA question piège)

La pipeline a 4 jobs. Les 3 premiers tournent sur les **runners hébergés de GitHub** (cloud). Le 4e (push-harbor) tourne sur un **runner self-hosted** (sur la machine).

Pourquoi ? Harbor est déployé **en local, non exposé sur Internet** : les serveurs cloud de GitHub ne peuvent pas l'atteindre. En entreprise on fait pareil : les contrôles sans état (lint, scan, signature) sur des runners cloud ; le **déploiement vers l'infrastructure interne** sur un runner installé dans le réseau privé. C'est un vrai patron d'architecture, pas un contournement. Ce job se déclenche **à la demande** (workflow_dispatch) quand le runner est en ligne.

3.6. Le fichier .github/workflows/ci.yml, job par job

Job	Runner	Étapes
lint	hébergé	hadolint (Dockerfile) + tflint + checkov (IaC). Bloquant.
sast	hébergé	Semgrep (règles p/default, p/python, p/flask). Bloquant.
build-scan-sign	hébergé	build de l'image → Trivy (exit-code 1) → ZAP baseline → Cosign sign + verify → push GHCR.
push-harbor	self-hosted	tire l'image de GHCR → pousse dans Harbor → déclenche le scan CVE de Harbor. À la demande.

Notions de syntaxe utiles : on: (quand se déclenche la pipeline), jobs: (les tâches), needs: (un job attend qu'un autre réussisse), runs-on: (sur quel runner), permissions: (droits du jeton, ex. id-token: write pour la signature keyless), steps: (les étapes), uses: (réutiliser une action), run: (lancer une commande).

3.7. Le parcours complet d'une image

git push → lint → SAST → build → Trivy (CVE, bloquant) → ZAP (DAST) → Cosign signe → push **GHCR** → (à la demande) le runner self-hosted tire l'image et la pousse dans **Harbor**, qui la re-scanne avec son propre Trivy → ces vulnérabilités alimentent le dashboard (Pilier 3).

 Platform Solutions Resources Open Source Enterprise Pricing

Search or jump to... / Sign in Sign up

KaTaKuRi-31 / harbor-devsecops Public

Notifications Fork 0 Star 0

<> Code Issues Pull requests **Actions** Projects Security and quality Insights

← CI DevSecOps — Harbor

✓ CI : login docker via -p sur le runner Windows (stdin PowerShell corr... #6 Sign in to view logs

Triggered via push June 22, 2026 10:57

Status

Total duration

Artifacts

 KaTaKuRi-31 pushed  main

Success

2m 54s

1

ci.yml

on: push

Annotations

3 warnings

 SAST (Semgrep)
Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4. For more information see: <https://github.bl-...>
[Show more](#)

 Lint (hadolint + tflint + checkov)
Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4, terraform-linters/setup-tflint@v4. For more...
[Show more](#)

 Build → Trivy → ZAP → Cosign → GHCR
Node.js 20 is deprecated. The following actions target Node.js 20 but are being forced to run on Node.js 24: actions/checkout@v4, actions/upload-artifact@v4, docker/login-ac...
[Show more](#)

Artifacts

Produced during runtime

Name	Size	Digest
 zap-baseline-report	18.7 KB	sha256:7dc6d9711cce366db595187ed85ac3b86eb1845a16739... 

Exécution GitHub Actions au statut « Success » : la pipeline bloquante a passé toutes les portes.

4. Pilier 3 — L'observabilité (Prometheus + Grafana)

4.1. Le principe

Observer un système = collecter ses **métriques** (chiffres mesurés dans le temps) pour connaître son état et détecter les problèmes. Ici, on surveille Harbor : son état, son activité, et surtout les **vulnérabilités (CVE)** qu'il détecte dans les images.

4.2. La chaîne de collecte

Composant	Rôle
Harbor	Expose des métriques (préfixe harbor_...) sur un endpoint /metrics, et scanne les images avec Trivy.
ServiceMonitor	Objet Kubernetes (CRD) qui dit à Prometheus : « viens collecter les métriques de Harbor ».
Prometheus	Collecte (scrape) et stocke les métriques dans le temps. Interrogeable via le langage PromQL.
Exporter CVE (maison)	Petit programme qui interroge l'API Harbor et expose le nombre de CVE par sévérité (harbor_image_vulnerabilities).
Grafana	Affiche les métriques dans un tableau de bord visuel (provisionné automatiquement).

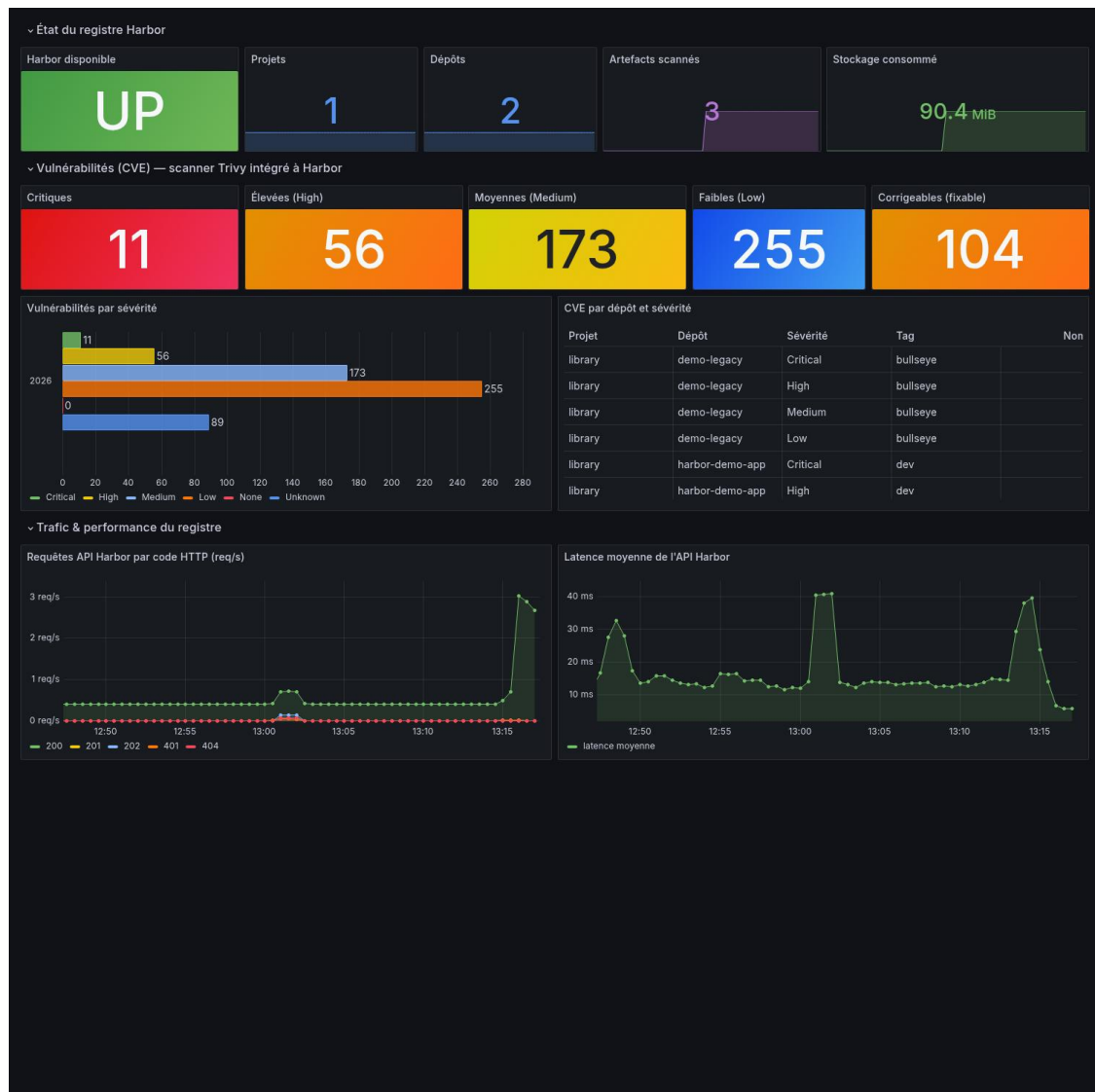
4.3. Pourquoi un exporter CVE « maison » ?

Les métriques natives de Harbor décrivent le **registre** (nombre de projets, dépôts, stockage, trafic) mais **pas le détail des CVE par sévérité**. On a donc ajouté un petit exporter (un programme Python déployé dans le cluster) qui interroge l'API de Harbor et expose des métriques comme `harbor_image_vulnerabilities{severity="Critical"}`. C'est la source du panneau « CVE » du dashboard.

Bon à savoir (le déploiement de l'exporter est « durci ») : utilisateur non-root, système de fichiers en lecture seule, capacités Linux supprimées, sondes de santé, limites de ressources. C'est ce qui lui permet de passer les contrôles **checkov** (sécurité Kubernetes) dans la CI.

4.4. Le dashboard Grafana « Harbor — Registre & CVE »

Trois zones : en haut l'état du registre (projets, dépôts, artefacts scannés, stockage) ; au milieu les **vulnérabilités par sévérité** (Critical, High, Medium, Low, corrigeables) et une table par dépôt ; en bas le trafic et la latence de l'API. Il est **provisionné automatiquement** (chargé via une ConfigMap repérée par le sidecar de Grafana).



Dashboard Grafana avec données réelles : compteurs de CVE par sévérité, état du registre et trafic.

4.5. Requêtes PromQL à connaître

harbor_up	# Harbor est-il scrappé ? (1 = oui)
harbor_statistics_total_repo_amount	# nombre de dépôts
harbor_image_vulnerabilities	# CVE par image et par sévérité (exporter)

5. Les autres fichiers clés à savoir expliquer

5.1. app/Dockerfile (l'image de démonstration)

C'est l'artefact qui traverse la pipeline. Il est **durci** pour passer hadolint : utilisateur **non-root** (appuser), dépendances **épinglées** (versions fixes), serveur de production **gunicorn**, et une sonde HEALTHCHECK. Base python:3.13-slim (récente → peu de CVE).

5.2. app/app.py

Une petite API Flask (3 routes : /, /healthz, /api/echo). Volontairement propre pour passer Semgrep ; elle sert de cible au DAST (ZAP) et de contenu à scanner/signer.

5.3. terraform/cve-exporter.tf

Déploie l'exporter CVE : une ConfigMap (le code Python), un Deployment durci, un Service, et un ServiceMonitor pour que Prometheus le collecte. Le mot de passe Harbor est lu depuis un **secret Kubernetes** (pas écrit en clair).

5.4. scripts/start-demo.ps1 et stop-demo.ps1

Démarrent / arrêtent la démo. stop-demo arrête le cluster (k3d) pour libérer la RAM tout en **conservant l'état** ; start-demo le relance et attend que tout soit prêt. Indispensable pour rallumer l'environnement avant l'oral.

6. Manipulations : « le prof demande... »

Avant tout : rallumer l'environnement avec powershell -ExecutionPolicy Bypass -File scripts\start-demo.ps1 (~3 min). Accès : Harbor localhost:30002 (admin / Harbor12345), Grafana localhost:30091 (admin / admin), Prometheus localhost:30090.

Demande du jury	Ce que tu fais
« Montre l'infrastructure as code »	Ouvrir terraform/main.tf (les 2 helm_release) ; terraform state list.
« Où sont les vulnérabilités ? »	UI Harbor → une image → onglet Vulnerabilities ; ou le dashboard Grafana CVE.
« Déclenche la pipeline »	git push (chaîne hébergée) ; ou bouton « Run workflow » pour inclure le push Harbor.
« Prouve que c'est bloquant »	trivy image sur l'app (renvoie 0) puis sur python:3.9-slim-bullseye (renvoie 1).
« Vérifie la signature »	cosign verify ... (identité = workflow GitHub).
« Explique les pods »	kubectl get pods -n harbor ; kubectl get pods -n monitoring.
« Montre que Harbor est surveillé »	kubectl get servicemonitor -n harbor ; Prometheus → Status → Targets.

6.1. Commandes de manipulation prêtes à l'emploi

```
# Cluster
kubectl get pods -n harbor
kubectl get servicemonitor -n harbor

# Vulnérabilités vues par Harbor (image ancienne, bien vulnérable)
curl -u admin:Harbor12345 \
  "http://localhost:30002/api/v2.0/projects/library/repositories/demo-
  legacy/artifacts/bullseye?with_scan_overview=true"

# Démontrer le blocage Trivy
docker run --rm -v //var/run/docker.sock:/var/run/docker.sock aquasec/trivy:latest \
  image --severity HIGH,CRITICAL --exit-code 1 python:3.9-slim-bullseye

# Vérifier la signature de l'image livrée par la CI
cosign verify ghcr.io/katakuri-31/harbor-devsecops/harbor-demo-app:latest \
  --certificate-identity-regexp "https://github.com/KaTaKuRi-31/harbor-devsecops/.*" \
  --certificate-oidc-issuer "https://token.actions.githubusercontent.com"
```

7. Questions / réponses probables du jury

« Pourquoi un runner self-hosted ? »

Parce que Harbor est déployé en local, non exposé sur Internet : les runners cloud de GitHub ne peuvent pas l'atteindre. On exécute donc les contrôles sans état sur des runners hébergés, et la publication vers le registre interne sur un runner installé dans le réseau privé. C'est le modèle standard en entreprise.

« Qu'est-ce qui rend la pipeline "bloquante" ? »

Chaque porte renvoie un code de sortie. Trivy avec `--exit-code 1` fait échouer le job (et donc la pipeline) dès qu'une CVE HIGH/CRITICAL corrigable est trouvée. L'image n'est alors pas publiée.

« Différence entre SAST et DAST ? »

Le SAST (Semgrep) analyse le code source sans l'exécuter (fautes de programmation). Le DAST (ZAP) teste l'application en cours d'exécution, de l'extérieur (en-têtes manquants, injections...). Les deux sont complémentaires.

« Pourquoi signer l'image, et pourquoi "keyless" ? »

Pour garantir l'origine et l'intégrité de l'image (sécurité de la chaîne d'approvisionnement). Keyless évite de gérer une clé privée : l'identité est le workflow GitHub (OIDC), le certificat vient de Fulcio, la preuve est dans le journal public Rekor.

« Comment Harbor détecte-t-il les CVE ? »

Harbor embarque le scanner Trivy. À chaque image poussée (ou à la demande), il la scanne et stocke le rapport de vulnérabilités, consultable dans l'UI ou via l'API.

« Comment les métriques arrivent-elles dans Grafana ? »

Harbor expose ses métriques ; un ServiceMonitor indique à Prometheus de les collecter ; un exporter maison ajoute le détail des CVE ; Grafana interroge Prometheus et affiche le tableau de bord.

« Pourquoi le depends_on dans Terraform ? »

Harbor crée un ServiceMonitor, qui est une CRD apportée par la stack Prometheus. Il faut donc installer Prometheus avant Harbor ; le `depends_on` force cet ordre.

« Que se passe-t-il si je relance terraform apply ? »

Rien ne change si l'infrastructure est déjà conforme (Terraform compare l'état souhaité à l'état réel). C'est l'idempotence : on peut réappliquer sans risque.

8. Glossaire express

Terme	Définition
IaC	Infrastructure as Code : décrire l'infra dans des fichiers versionnés.
Helm / chart	Gestionnaire de paquets de Kubernetes / un paquet paramétrable.
Namespace	Cloison logique isolant des ressources dans le cluster.
CRD	Custom Resource Definition : nouveau type d'objet ajouté à Kubernetes (ex. ServiceMonitor).
Registre	Entrepôt d'images Docker (Harbor, GHCR...).
SAST / DAST	Analyse de sécurité statique (code) / dynamique (appli en exécution).
CVE	Vulnérabilité publique répertoriée.
Lint	Vérification automatique de qualité/bonnes pratiques d'un fichier.
Cosign / keyless	Signature d'image / sans clé privée (identité = workflow via OIDC).
OIDC	Protocole d'identité prouvant « ce workflow GitHub est bien lui ».
Fulcio / Rekor	Autorité de certificats éphémères / journal public de transparence (Sigstore).
GHCR	GitHub Container Registry : le registre d'images de GitHub.
ServiceMonitor	Objet disant à Prometheus quelles cibles collecter.
PromQL	Langage de requêtes de Prometheus.
Runner	Machine qui exécute les jobs GitHub Actions (hébergée ou self-hosted).
exit-code	Code de sortie d'une commande ; 0 = succès, ≠0 = échec (bloque la CI).

Dernier réflexe pour l'oral : si tu ne sais pas répondre, ouvre le fichier concerné et raisonne à voix haute. Montrer qu'on sait **où chercher** et comprendre la logique vaut mieux que réciter par cœur.